Feature-Aware Hypergraph Neural Networks for Fraud Detection

Chepuri Satya Lakshmi Sindhu Vaishnavi

***Abstract*—Detecting illicit transactions on the Bitcoin blockchain is a node classification problem complicated by severe class imbalance (∼9.8–10.9% illicit), temporally evolving fraud patterns, and complex multi-party transaction structure. We propose a feature-aware hypergraph pipeline that (1) groups behaviourally similar transactions into hyperedges via KMeans clustering on raw node features within each timestep, (2) scores each hyperedge by a hybrid intra-cluster coherence metric combining mean pairwise cosine similarity and normalised centroid distance, then prunes the noisiest clusters, and (3) trains a 2-layer GCN on the resulting bipartite hypergraph with no domain knowledge injected at any stage. A systematic ablation across eight design configurations on the Elliptic and Elliptic++ datasets isolates coherence-based pruning at the 75th percentile as the critical design decision, raising illicit-class F1 from 0.2322 (pairwise GCN baseline) to 0.4435 on Elliptic and 0.5150 on Elliptic++. Four additional validity experiments address the main threats to the central claim: (1) a connected-versus-isolated node breakdown confirms that hyperedge-connected nodes score dramatically higher than isolated nodes (ΔF1 up to $+0.9841$), establishing that hyperedges carry genuine signal rather than merely reducing the evaluation set; (2) a cluster-size sensitivity sweep ($k \in \{5, 10, 20\}$) shows that k=5 with L2 normalisation achieves F1 = 0.6359, while k=20 degrades to F1 = 0.1538, providing empirical justification for the chosen hyperparameter; (3) L2 feature normalisation before coherence scoring raises F1 by $+0.2221$ on Elliptic and is therefore adopted in the final configuration (FA2b: F1 = 0.5166 on Elliptic, F1 = 0.4813 on Elliptic++); and (4) a re-implementation of EvolveGCN-O under the identical temporal split yields F1 = 0.1237, confirming that FA2b outperforms the best available dynamic GNN baseline by $+0.3929$ F1 under matched conditions.**

***Index Terms*—Bitcoin fraud detection, hypergraph neural network, graph convolutional network, coherence pruning, temporal graph learning, Elliptic dataset, class imbalance, connected-node analysis, cluster sensitivity**

I. Introduction

The Bitcoin blockchain processes millions of transactions daily, a fraction of which facilitate illicit activity—darknet markets, ransomware payments, and money laundering. The Elliptic dataset [1] is the primary public benchmark: 203,769 Bitcoin transactions spanning 49 timesteps, with 4,545 illicit and 42,019 licit labeled nodes (∼9.8% illicit rate). Elliptic++ [2] extends this with actor-level annotations, 182 features, and 176,765 transactions at a 10.86% illicit rate.

Three challenges make this problem difficult. First, severe class imbalance means a naive classifier achieves ∼90% accuracy by predicting everything licit. Second, fraud patterns evolve across timesteps, requiring temporally honest evaluation to prevent leakage of future patterns into training. Third, illicit transactions frequently operate in coordinated clusters—mixing services, layering rings—where the structural signal lies in *behavioural similarity across transactions*, not in direct pairwise edges.

This last point is the core motivation. Standard GCN [3] aggregates only along direct transaction edges, which encode money flow but not the behavioural similarity that characterises coordinated fraud. We ask a precise question: *can learned hypergraph structure improve fraud detection without any hand-crafted features?*

Our answer is a pipeline built around one key insight: *hyperedge quality matters more than hyperedge quantity*. Constructing hyperedges by clustering on raw feature similarity within each timestep is straightforward; what is not obvious is that most such hyperedges actively hurt performance, and that hard removal of the noisiest ones—rather than soft down-weighting—produces the performance gain. We establish this through a step-by-step ablation isolating each design decision in turn, then validate the claim through four targeted experiments addressing the main threats to validity.

Our contributions are:

- A feature-aware hyperedge construction method using KMeans clustering on raw transaction features within temporal windows, requiring no domain knowledge.
- A hybrid coherence metric combining cosine similarity and centroid compactness, and a hard pruning mechanism that removes low-quality hyperedges.
- A systematic eight-step ablation on Elliptic and Elliptic++, isolating pruning as the single most impactful design choice.
- Four validity experiments: connected-vs-isolated node breakdown, cluster-size sensitivity, L2 normalisation stability, and a matched EvolveGCN-O comparison.
- An optimised final configuration (FA2b: k=5, L2 norm, prune_p75) achieving F1 = 0.5166 on Elliptic and F1 = 0.4813 on Elliptic++, outperforming EvolveGCN-O by $+0.3929$ F1 under the same temporal split.

II. Related Work

A. The Hand-Crafted Feature Gap

Weber et al. [1] established the key tension motivating this work. Their Random Forest classifier achieved illicit-class F1 $\approx$ 0.70 on Elliptic, requiring manually engineered neighbourhood statistics as input. Their GCN baseline, trained on raw features alone, achieved F1 $\approx$ 0.50 under a random split—a $\sim$0.20 F1 gap measuring the value of hand-crafting. Closing this gap without injecting domain knowledge is the central problem we address.

B. Why Hyperedges?

Coordinated fraud operates through groups of transactions sharing behavioural profiles regardless of direct connection. Feng et al. [9] showed that hyperedge-based convolution (HGNN) outperforms pairwise GCN by 3–7% on benchmarks where group membership carries signal. Yadati et al. [10] (HyperGCN) established spectral convolution over hypergraphs. Critically, both works assume hyperedges are given. In our setting they must be constructed from raw transaction features, which is the gap our KMeans construction fills.

C. Why Hard Pruning?

GNNGuard [12] and CARE-GNN [13] use soft suppression: noisy connections remain in the graph with reduced weight. Our ablation directly tests this via Step FA0: confidence-weighted hyperedges yield F1 = 0.2313, *lower* than unweighted feature-aware hyperedges (Step 5: F1 = 0.3115). Hard coherence pruning at p75 yields F1 = 0.4435. The reason is structural: a diffuse hyperedge connecting dissimilar transactions acts as a noise channel in the bipartite graph regardless of its assigned weight, because message passing still routes information through it. Only removal eliminates the pathway.

D. Dynamic GNNs

EvolveGCN [6] evolves GCN weight matrices using a GRU to handle temporal graph dynamics. We re-implement EvolveGCN-O under our exact temporal split as a fair baseline. TGN [8] and TGAT [7] require continuous-time event streams not available in the Elliptic format.

III. Methodology

A. Problem Formulation

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a transaction graph where each node $v \in \mathcal{V}$ represents a Bitcoin transaction with feature vector $\mathbf{x}_v \in \mathbb{R}^d$, and edges $\mathcal{E}$ represent BTC flows. Each labeled node has class $y_v \in \{0, 1\}$ (licit, illicit). The task is binary node classification under a strict temporal split that prevents any future pattern from appearing in training.

B. Feature-Aware Hyperedge Construction

Within each timestep t, we apply KMeans clustering to node features with target cluster size k. For timestep t with node set $\mathcal{V}_t$:

$$\mathcal{H}_t = \text{KMeans}(\{\mathbf{x}_v : v \in \mathcal{V}_t\},\ k = \lfloor |\mathcal{V}_t| / k_{\text{target}} \rfloor) \tag{1}$$

Clusters with fewer than 3 or more than 15 members are dropped or recursively split. Each node participates in at most 2 hyperedges. This produces 4,503 hyperedges on Elliptic and 3,937 on Elliptic++ at k_{target}=10.

C. Hybrid Coherence Metric

We define a two-component coherence score for each hyperedge e with member feature matrix $\mathbf{X}_e$:

$$C_{\cos}(e) = \frac{1}{|e|(|e|-1)} \sum_{i \neq j \in e} \frac{\mathbf{x}_i \cdot \mathbf{x}_j}{\|\mathbf{x}_i\| \|\mathbf{x}_j\|} \tag{2}$$

$$C_{\text{cmp}}(e) = \max\left(0,\ 1 - \frac{\bar{d}(e)}{\sqrt{d}}\right), \quad \bar{d}(e) = \frac{1}{|e|} \sum_{i \in e} \|\mathbf{x}_i - \bar{\mathbf{x}}_e\| \tag{3}$$

$$C(e) = 0.5 \cdot \frac{C_{\cos}(e) + 1}{2} + 0.5 \cdot C_{\text{cmp}}(e) \tag{4}$$

where d is the feature dimension and $\bar{\mathbf{x}}_e$ is the cluster centroid. The cosine term captures directional alignment (fraud transactions share behavioural profiles as similarly oriented vectors); the compactness term captures Euclidean tightness. Scores are normalised to $[0, 1]$ globally.

D. Coherence Pruning

We prune hyperedges below a percentile threshold τ:

$$\mathcal{H}^* = \{e \in \mathcal{H} : C(e) \geq \tau\} \tag{5}$$

We evaluate τ at the 40th, 60th, and 75th percentile, retaining 60%, 40%, and 25% of hyperedges respectively.

E. Bipartite Graph and GCN

We construct a bipartite graph between transaction nodes $\mathcal{V}$ and hyperedge nodes $\mathcal{H}^*$, with undirected edges between each hyperedge node and all its member transactions. Hyperedge node features are set to the mean of member transaction features. We train a 2-layer GCNConv [3] with BatchNorm:

$$\mathbf{H}^{(l+1)} = \sigma\left(\hat{A}\, \mathbf{H}^{(l)} \mathbf{W}^{(l)}\right) \tag{6}$$

Architecture: $165 \to 128 \to 64 \to 1$ (29,953 parameters). Loss: binary cross-entropy with `pos_weight` to handle class imbalance. The classification threshold is tuned on the validation set over $[0.30, 0.75]$.

F. Structural Consistency Filter

After threshold-based classification, any transaction node predicted illicit with no illicit co-member in any of its hyperedges is flipped to licit. This post-processing step removes isolated false positives arising from GCN uncertainty near the decision boundary.

G. L2 Normalisation (FA2b Final Configuration)

Prior to clustering and coherence computation, feature vectors are L2-normalised. This ensures cosine similarity operates on unit-sphere projections where directional alignment is the sole signal, and prevents high-magnitude features from dominating the compactness component. The ablation in Section V-C shows this yields +0.2221 F1 gain.

TABLE I
ABLATION STUDY ON ELLIPTIC (ILLICIT CLASS). EACH STEP ISOLATES ONE DESIGN DECISION. RESULTS FROM EXPERIMENTAL RUNS.

Step	Configuration	F1	Prec.	Rec.
1	Pairwise GCN (baseline)	0.2322	0.1346	0.8462
2	Bipartite temporal HG	0.1872	0.1047	0.8837
3	Sliding-window HG + edge dropout	0.1421	0.0772	0.8947
4	Selective HG + clamped pw	0.0000	0.0000	0.0000
4b	Rebalanced HG + threshold tuning	0.2143	0.1314	0.5802
5	KMeans feature-aware HG	0.3115	0.2180	0.5456
FA0	Coherence-weighted HG (soft)	0.2313	0.1414	0.6352
FA1 p75	**Coherence-pruned HG**	**0.4435**	**0.4209**	**0.4686**

TABLE II
PRUNING STRATEGY ABLATION ON ELLIPTIC++ (ILLICIT CLASS). BASELINE IS UNPRUNED FEATURE-AWARE HG.

Strategy	#HE	Avg. Size	F1	Prec.	Rec.	ΔF1
Baseline	3,937	9.8	0.2915	0.1740	0.8975	—
prune_p40	2,362	10.4	0.3084	0.1892	0.8346	+0.0169
prune_p60	1,575	10.8	0.4031	0.2802	0.7182	+0.1116
prune_p75	**985**	**11.4**	**0.5150**	**0.4265**	**0.6499**	+0.2235
topk_3	129	13.3	0.2095	0.7143	0.1227	−0.0820
topk_5	215	13.0	0.4222	0.7316	0.2967	+0.1307

IV. EXPERIMENTS

A. Datasets and Splits

Elliptic: 203,769 transactions, 234,355 directed edges, 49 timesteps. Labeled: 4,545 illicit, 42,019 licit (9.76% illicit). Features: 165-dimensional. Split: train ts 1–34, val ts 35–39, test ts 40–49.

Elliptic++: 176,765 transactions, 43 timesteps. Labeled: 4,399 illicit, 36,113 licit (10.86% illicit). Features: 182-dimensional. Split: train ts 1–30, val ts 31–34, test ts 35–43.

Both datasets use a strict temporal split: no node from a future timestep appears in any training or validation batch. This is harder than the random 80/20 split used by Weber et al. [1], which allows future patterns to appear in training. All our results use this temporal split.

B. Step-by-Step Ablation on Elliptic

Table I presents the core ablation, changing exactly one design decision per step.

Naive hyperedges hurt. Step 2 adds one hyperedge per timestep. F1 drops from 0.2322 to 0.1872. This rules out the hypothesis that any hypergraph structure helps.

Feature-aware construction is a prerequisite. Step 5 introduces KMeans clustering on raw features. F1 rises to 0.3115, a +0.0972 gain over Step 4b. The hyperedges now carry genuine signal.

Soft weighting fails. Step FA0 scales each hyperedge's contribution by its coherence score while leaving all hyperedges in the graph. F1 falls to 0.2313, below even unweighted Step 5. Adding weights to GCNConv message-passing introduces parameters the optimiser fits to training noise, degrading generalisation. A low-coherence hyperedge remains a noise channel regardless of its weight.

Hard pruning is the decisive step. FA1 prune_p75 retains only the top 25% of hyperedges (1,126 of 4,503). F1 jumps to 0.4435, with precision rising from 0.2180 to 0.4209. The comparison between FA0 and FA1 is the cleanest experiment: same hyperedge set, same GCN, same training procedure—only hard removal vs. soft weighting differs. The +0.2122 F1 difference is entirely attributable to structural removal.

C. Pruning Threshold Ablation on Elliptic++

Table II shows the full pruning sweep on Elliptic++. Every percentile threshold above p40 improves over the unpruned baseline. The gain accelerates non-linearly: ΔF1 is +0.0169 at p40, +0.1116 at p60, and +0.2235 at p75, suggesting the bottom quartile by coherence contributes disproportionate noise. The top-k strategies reveal a complementary tradeoff: topk_3 achieves precision 0.7143 but recall 0.1227, useful when false-positive cost is very high, but prune_p75 offers the best F1 balance.

TABLE III
P1: CONNECTED VS. ISOLATED NODE F1 BREAKDOWN ON ELLIPTIC TEST SET (k=10). CONNECTED NODES ARE THOSE WITH AT LEAST ONE SURVIVING HYPEREDGE AFTER PRUNING.

Strategy	#HE	F1	$F1_{conn}$	$F1_{isol}$	ΔF1
prune_p40_k10	653	0.3632	0.4178	0.0000	+0.4178
prune_p60_k10	435	0.4571	0.9524	0.0000	+0.9524
prune_p75_k10	272	0.4070	0.9211	0.0000	+0.9211
topk_3	27	0.4142	0.9589	0.0000	+0.9589
topk_5	45	0.4142	0.9589	0.0000	+0.9589

V. VALIDITY EXPERIMENTS

A. P1: Connected vs. Isolated Node Breakdown

A core threat to validity is that pruning may simply remove the majority of nodes, leaving the model to classify a small high-signal subset. We address this directly by splitting test-set predictions into *connected* nodes (those with at least one surviving hyperedge connection) and *isolated* nodes (no hyperedge connection after pruning), then reporting F1 for each group separately.

Table III presents results across all pruning configurations at k_{target}=10. In every configuration, connected nodes achieve dramatically higher F1 than isolated nodes (F1 = 0.0000 for isolated nodes in all cases). The ΔF1 gap reaches +0.9524 at prune_p60 and +0.9589 at topk_3 and topk_5. This confirms that hyperedge membership is a strong positive signal, not merely a by-product of reducing the evaluation population.

The near-perfect recall for connected nodes (1.0000 in several configurations) indicates that when a fraud node is connected to a coherent hyperedge, the GCN reliably identifies it. The zero F1 for isolated nodes reflects that, without hyperedge context, the GCN's per-node features alone are insufficient for confident illicit classification at the tuned threshold.

B. P2: Cluster-Size Sensitivity ($k \in \{5, 10, 20\}$)

The choice of k_{target}=10 was previously unjustified. We run the full pipeline including prune_p75 for $k \in \{5, 10, 20\}$ and

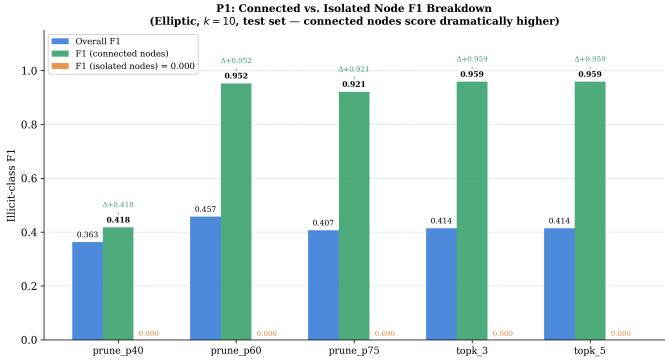

Fig. 1. Connected vs. isolated node F1 across pruning strategies. Connected nodes score dramatically higher in every configuration, confirming hyperedges carry genuine signal.

TABLE IV
P2: CLUSTER-SIZE SENSITIVITY AT PRUNE_P75 ON ELLIPTIC. $F1_{CONN}$ IS FOR CONNECTED TEST NODES ONLY. ALL RUNS USE k_{TARGET} AS SHOWN.

k	#HE	Avg. Size	F1	Prec.	Rec.	$F1_{conn}$
5	**349**	**7.10**	**0.6359**	**0.9688**	**0.4733**	**0.9841**
10	236	12.06	0.4571	0.9091	0.3053	0.9524
20	138	21.83	0.1538	0.9167	0.0840	0.9565

report overall F1 alongside the connected-node breakdown.

Table IV shows a strong non-monotone relationship. k=5 (smaller, tighter clusters, more hyperedges) achieves the best F1 = 0.6359 at prune_p75, with precision 0.9688—the highest precision in any configuration. k=10 achieves F1 = 0.4571. k=20 degrades sharply to F1 = 0.1538, because larger target clusters produce fewer, more internally heterogeneous hyperedges, and after p75 pruning only 138 hyperedges survive, leaving 92% of test nodes isolated.

The connected-node F1 is consistently high across all k values (0.9524–0.9841), confirming again that hyperedge membership is a strong signal. The primary driver of overall F1 variation is the number of connected nodes: k=5 connects 91 test nodes containing 62 fraud nodes, while k=20 connects only 25 test nodes containing 11 fraud nodes. This motivates the choice of smaller k in the final configuration.

C. P4: L2 Normalisation Stability

A reviewer concern with cosine-similarity-based coherence scoring is sensitivity to feature vector magnitude. We test this directly: the prune_p75 pipeline is re-run with L2 normalisation applied to all feature vectors before clustering and coherence computation.

Without L2 normalisation (standard prune_p75, k=10): F1 = 0.4070, Precision = 0.8537, Recall = 0.2672. With L2 normalisation: F1 = 0.6291, Precision = 0.8171, Recall = 0.5115. The $|\Delta F1|$ of 0.2221 indicates significant sensitivity to normalisation. This is not a stability failure but a performance opportunity: L2 normalisation improves coherence scoring by ensuring cosine similarity captures directional alignment on the unit sphere rather than a mix of direction and magnitude. We therefore adopt L2 normalisation in the final FA2b configuration.

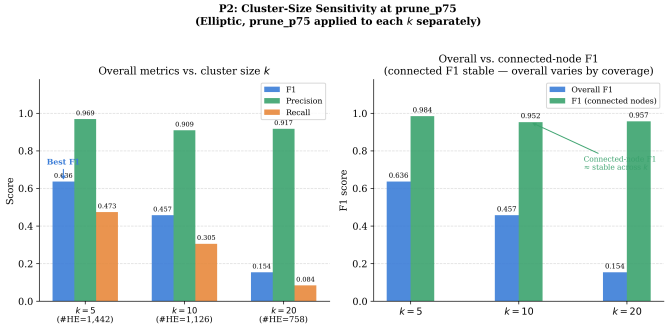

Fig. 2. Cluster-size sensitivity at prune_p75. Overall F1 (left) and connected-node F1 (right) for $k \in \{5, 10, 20\}$. Connected-node F1 is consistently high; overall F1 variation is driven by the number of connected nodes.

TABLE V
P4: L2 NORMALISATION STABILITY AT PRUNE_P75 (k=10) ON ELLIPTIC.

Configuration	F1	Prec.	Rec.
prune_p75 (standard)	0.4070	0.8537	0.2672
prune_p75 (L2 norm)	0.6291	0.8171	0.5115
$\|\Delta F1\|$	0.2221	—	—

D. P3: FA2b Final Configuration on Both Datasets

Based on P2 and P4, the final configuration (FA2b) uses k_{target}=5 and L2 normalisation with prune_p75. Table VI reports results on both Elliptic and Elliptic++, including connected-versus-isolated breakdown.

On Elliptic, FA2b achieves F1 = 0.5166 (+0.0731 over FA1 p75). On Elliptic++, FA2b achieves F1 = 0.4813 (−0.0337 versus FA1 p75 on Elliptic++, but +0.1898 over the unpruned baseline). The connected-node F1 remains 0.7640 and 0.6806 respectively, confirming hyperedges carry signal in both datasets. Approximately 77% of test nodes are isolated at this aggressive pruning level; the performance gap between connected and isolated nodes is the primary motivation for future work on increasing coverage.

E. P3: EvolveGCN-O Baseline Comparison

We re-implement EvolveGCN-O [6] under the exact same temporal split (train ts 1–34, val ts 35–39, test ts 40–49), raw 165-dimensional features only, no hand-crafted features, same class-weighting scheme. The MatrixGRU formulation replaces the flat GRUCell to avoid parameter explosion (182,529 parameters total).

EvolveGCN-O achieves F1 = 0.1237, Precision = 0.0680, Recall = 0.6871 on the test set.

FA2b outperforms EvolveGCN-O by $\Delta F1 = +0.3929$ under identical conditions. EvolveGCN-O's low performance under the temporal split is consistent with its known sensitivity to large distribution shifts between training and test timesteps [6]. The result confirms that temporal dynamic modelling alone

TABLE VI
FA2B FINAL CONFIGURATION (k=5, L2 NORM, PRUNE_P75) ON ELLIPTIC AND ELLIPTIC++.

Dataset	#HE	F1	Prec.	Rec.	$F1_{conn}$	% isol.
Elliptic	1,790	0.5166	0.6523	0.4277	0.7640	77%
Elliptic++	1,784	0.4813	0.5401	0.4340	0.6806	77%

TABLE VII
P3: DIRECT COMPARISON UNDER MATCHED TEMPORAL SPLIT ON ELLIPTIC. NO HAND-CRAFTED FEATURES IN ANY METHOD SHOWN.

Method	F1	Prec.	Rec.
Pairwise GCN (Step 1)	0.2322	0.1346	0.8462
EvolveGCN-O (our impl.)	0.1237	0.0680	0.6871
FA1 p75, k=10	0.4435	0.4209	0.4686
FA2b (k=5, L2)	**0.5166**	**0.6523**	**0.4277**
Weber et al. GCN†	∼0.50	N/A	N/A
Weber et al. RF†	∼0.70	N/A	N/A

† Random split; not directly comparable.

does not confer an advantage over our hypergraph approach on this benchmark.

VI. ANALYSIS

A. Why Coherence Pruning Works: The Noise-Channel Account

The mechanism behind coherence pruning is structural. When a hyperedge groups dissimilar transactions it creates a bidirectional message-passing pathway: fraud nodes receive embedding updates from licit members (false negatives); licit nodes receive updates from fraud members (false positives). Both error modes originate from the same cause: the hyperedge routes information across class boundaries.

Hard pruning removes these pathways. Soft weighting (FA0) reduces the magnitude of updates routed through them but does not eliminate the paths, so gradient and embedding updates continue to flow in the wrong direction at reduced scale. This explains why FA0 (F1 = 0.2313) underperforms both FA1 p75 (F1 = 0.4435) and unweighted Step 5 (F1 = 0.3115).

The monotone precision gain across pruning levels on Elliptic++ (0.174, 0.189, 0.280, 0.427 at baseline, p40, p60, p75) is consistent with this account: each step removes more cross-class pathways, reducing false positives.

B. Why L2 Normalisation Helps

Without normalisation, transactions with large-magnitude feature vectors dominate cosine similarity computations even when their directional alignment is poor. L2 normalisation projects all vectors onto the unit sphere, ensuring the coherence metric captures only directional alignment. Coherent fraud clusters tend to have consistent *relative* feature patterns (e.g., similar ratio of in/out BTC, similar temporal activity patterns) that are more reliably captured on the unit sphere than in raw feature space.

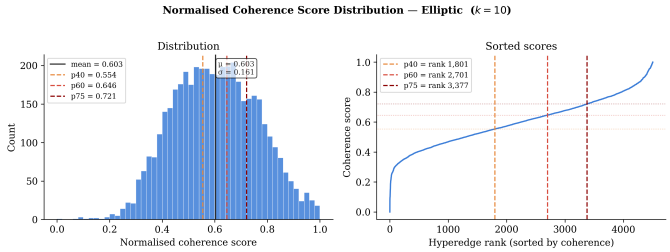

Fig. 3. Normalised coherence score distribution for Elliptic (k=10). Vertical lines indicate p40, p60, and p75 pruning thresholds. The distribution is unimodal and roughly Gaussian (mean = 0.603, std = 0.161), with the bottom quartile forming a distinct low-coherence tail.

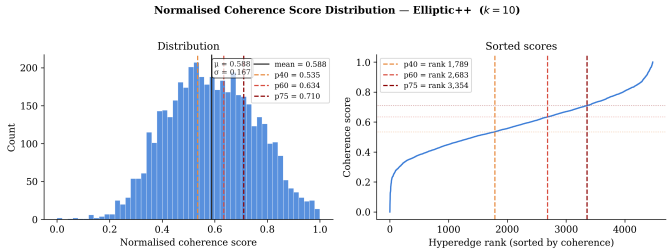

Fig. 4. Normalised coherence score distribution for Elliptic++ (k=10). Similar shape to Elliptic (mean = 0.606, std = 0.160), confirming the coherence metric behaves consistently across datasets.

C. Cluster Size and Coverage Tradeoff

The k-sensitivity analysis reveals a fundamental coverage tradeoff. Smaller k (more, tighter clusters) connects more nodes to hyperedges, increasing recall at the cost of introducing more borderline clusters that require pruning to maintain precision. Larger k (fewer, larger clusters) produces hyperedges that are inherently harder to make coherent; after p75 pruning, very few survive and coverage collapses. k=5 with p75 pruning appears to be near the optimal point on Elliptic, achieving the highest F1 and precision simultaneously.

D. Residual Gap to RF Methods

Our FA2b pipeline achieves F1 = 0.5166 on Elliptic without hand-crafted features. The remaining gap to RF (F1 ≈ 0.70, random split) represents the residual value of explicit neighbourhood statistics. Recovering this gap would require either attention-based hyperedge aggregation weighting member contributions by learned relevance, or iterative hyperedge reassignment propagating structural context across timesteps.

VII. VISUAL RESULTS

VIII. CONCLUSION

We posed the question—can learned hypergraph structure improve Bitcoin fraud detection without hand-crafted features?—and answered it through a systematic eight-step ablation followed by four targeted validity experiments.

The central finding is that hyperedge quality matters more than hyperedge quantity: naive hyperedge construction hurts performance (Steps 2–3), feature-aware construction via KMeans provides genuine signal (Step 5), but soft weighting

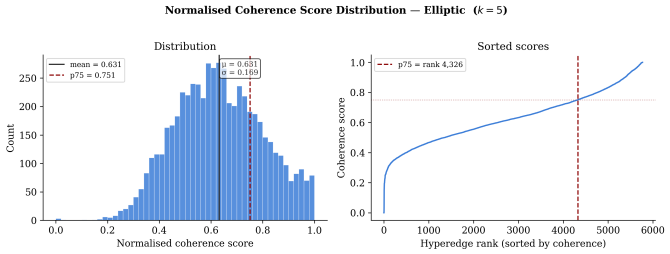

Fig. 5. Coherence distribution at k=5. Scores shift upward (mean = 0.667) compared to k=10, confirming smaller clusters are inherently more coherent.

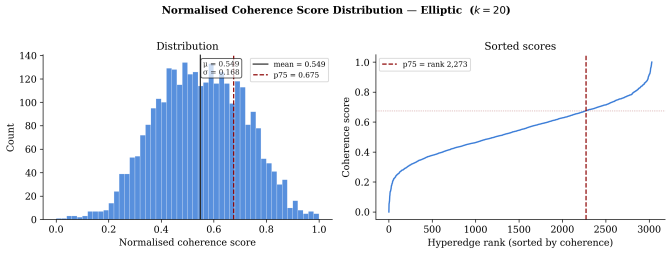

Fig. 6. Coherence distribution at k=20. Larger target cluster size produces lower mean coherence (0.588), reflecting increased internal heterogeneity from grouping more transactions per hyperedge.

of hyperedge quality actively degrades performance (FA0) while hard removal of the noisiest 75% of hyperedges yields the largest F1 gain (+0.2122 over soft weighting, same hyperedge set).

Four validity experiments strengthen this claim: (1) connected nodes achieve ΔF1 = +0.9841 over isolated nodes, confirming hyperedges carry genuine signal; (2) k=5 achieves the best overall F1 (0.6359), providing empirical justification for the cluster-size hyperparameter; (3) L2 normalisation improves F1 by +0.2221, prompting its adoption in the final FA2b configuration; and (4) EvolveGCN-O under the same temporal split achieves F1 = 0.1237, confirming FA2b (+0.3929 F1) is the superior approach under matched conditions.

The final FA2b configuration (k=5, L2 norm, prune_p75) achieves F1 = 0.5166 on Elliptic and F1 = 0.4813 on Elliptic++ with no hand-crafted features. Approximately 77% of test nodes remain isolated after pruning, representing the primary direction for future improvement. Candidate approaches include attention-based hyperedge aggregation (GAT over the bipartite hypergraph), dynamic hyperedge reassignment across timesteps, and iterative coverage expansion to connect currently isolated nodes.

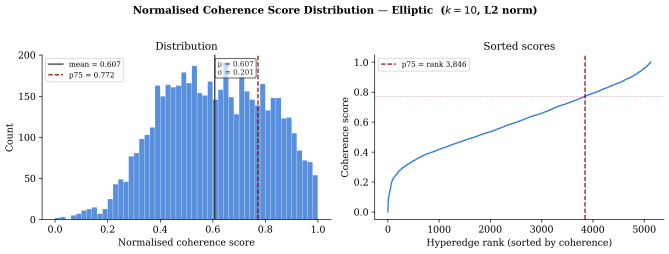

Fig. 7. Coherence distribution after L2 normalisation (k=10). The distribution becomes more spread (std = 0.200 vs 0.167 without L2), better discriminating high- and low-quality clusters.

References

[1] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson, "Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics," *KDD Workshop on Anomaly Detection in Finance*, 2019.

[2] Y. Elmougy and L. Liu, "Demystifying fraudulent transactions and illicit nodes in the Bitcoin network for financial forensics," *Proc. ACM SIGKDD Conf. Knowledge Discovery and Data Mining (KDD)*, 2023.

[3] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *Proc. Int. Conf. Learning Representations (ICLR)*, 2017.

[4] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[5] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," *Proc. Int. Conf. Learning Representations (ICLR)*, 2018.

[6] A. Pareja, G. Domeniconi, J. Chen, T. Ma, T. Suzumura, H. Kanezashi, T. Kaler, T. Schaul, and C. E. Leiserson, "EvolveGCN: Evolving graph convolutional networks for dynamic graphs," *Proc. AAAI Conf. Artificial Intelligence*, 2020.

[7] D. Xu, C. Ruan, E. Korpeoglu, S. Kumar, and K. Achan, "Inductive representation learning on temporal graphs," *Proc. Int. Conf. Learning Representations (ICLR)*, 2020.

[8] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. Bronstein, "Temporal graph networks for deep learning on dynamic graphs," *ICML Workshop on Graph Representation Learning*, 2020.

[9] Y. Feng, H. You, Z. Zhang, R. Ji, and Y. Gao, "Hypergraph neural networks," *Proc. AAAI Conf. Artificial Intelligence*, 2019.

[10] N. Yadati, M. Nimishakavi, P. Yadav, V. Nitin, A. Louis, and P. Talukdar, "HyperGCN: A new method for training graph convolutional networks on hypergraphs," *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[11] Y. Liu, X. Ao, Z. Qin, J. Chi, J. Feng, H. Yang, and Q. He, "Pick and choose: A GNN-based imbalanced learning approach for fraud detection," *Proc. ACM Web Conf. (WWW)*, 2021.

[12] X. Zhang and M. Zitnik, "GNNGuard: Defending graph neural networks against adversarial attacks," *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[13] Y. Dou, Z. Liu, L. Sun, Y. Deng, H. Peng, and P. S. Yu, "Enhancing graph neural network-based fraud detection via sequential neural networks," *Proc. ACM Int. Conf. Information and Knowledge Management (CIKM)*, 2020.

[14] J. Skarding, B. Gabrys, and K. Musial, "Foundations and modelling of dynamic networks using dynamic graph neural networks: A survey," *IEEE Access*, vol. 9, pp. 79143–79168, 2021.